AI8：基于 LSH 的近似最近邻检索

A 问题是什么

给定一个查询向量，怎样在大量向量中快速找到最相似的几个？逐个计算距离能得到精确结果，但数据增大后可能很慢。LSH（Locality-Sensitive Hashing，局部敏感哈希）让相近向量更容易落入同一桶，先筛候选，再精确重排。Basic 建一张表并与暴力搜索核对；Advance 用多表、参数实验和标准答案测量召回、速度与内存的折中。

阶段	工作量安排	难度	完成后的能力
Basic	一周内约 10-20 小时	easy	建桶、查桶、候选去重与距离排序
Advance	完成 Basic 后再用六周	medium	实现多表索引，公平测量 Recall@K 与完整查询成本

本说明以**欧氏距离的 E2LSH** 为主线，Python/NumPy 即可。v0/25-LSH 同时含 E2LSH 说明和随机超平面 C++ 实现，二者哈希族不同；可参考文件组织和数据，但不要把原始像素上的随机超平面直接当成 E2LSH。参考 v0 LSH 说明。

Basic：先缩小候选范围，再算真实距离

1 学习目标与需要回答的问题

提交 Q1-Q5，Q5 附桶号和真实距离。

编号	需要回答的问题	回答应包含的内容
Q1	精确与近似最近邻有何区别？	近似检索可能漏掉全局近邻，用召回衡量漏检
Q2	LSH 与普通哈希目标有何不同？	相似点希望高概率碰撞，不能期待所有不同点分开
Q3	为什么桶内还要精确算距离？	同桶不保证近；精确重排只保证候选内部次序
Q4	Recall@K 与分类准确率相同吗？	前者看找回多少真实近邻 ID，后者看类别是否相同
Q5	h(x)=floor(x/2)，q=1.9，下表哪些点会入候选？	找出真实最近邻和同桶最近邻，解释漏检

ID	一维数据 x
a	0.2
b	1.0
c	2.1
d	4.5

2 本周用到的哈希（先读这段）

把高维投影到数轴，再切格子。 E2LSH 选择各维独立标准高斯的向量 a，以及均匀偏移 b，使用：

```
h(x) = floor((a· x + b) / w)
a：随机投影向量；b∈[0,w)；w>0：桶宽
```

相近点更可能同桶，但在边界两侧也会分开。随机偏移使格子边界变化，不能消除所有漏检。

多次哈希形成一个键。 一张表可用 k 个哈希值组成元组，全部相同才同桶；这是一次候选筛选，不是计算“近似距离”。

库向量与查询分开。 索引只存库向量 ID；查询用建表时同一组投影，不能临时重新抽样。Basic 的手算固定 a、b，只为核对运算，不代表随机族的统计性质。

编号	材料	建议用法
B1	NumPy 入门	点积、数组和随机数
B2	Gionis 等：Similarity Search via Hashing	理解相似检索、碰撞和多表思路
B3	v0 实现指南	看建表、ID、候选去重；注意其中采用随机超平面

3 代码任务：一张表的最小检索器

1. 自行生成 100-1000 个二维或低维向量，固定种子。
2. 编写暴力 top-K：对全库精确算距离，按距离和 ID 排序。
3. 实现 E2LSH 哈希与字典桶，Basic 先取 k=1 或 2。
4. 查询同键桶，得到候选 ID，再按真实欧氏距离排序。
5. 空桶返回空结果；候选不足 K 就返回实际数量，不能悄悄全库扫描。

建议自行编写以下接口，**这些是作业设计，不是已存在的 Python 文件**：

```
build_index(vectors, k, w, seed)
query(index, query_vector, top_k)
brute_force(vectors, query_vector, top_k)
```

检查负数的 floor，例如 floor(-0.1)=-1，不能用向零截断替代。相同输入的键应固定，桶中总 ID 数等于库大小。自查询包含自己可用于实现核对，正式评价使用独立查询，或明确排除自身 ID。

4 用一个小例子核对

q=1.9 的桶号为 0，固定 a=1,b=0,w=2。

ID	x	桶号	到 q 的距离	同桶候选
a	0.2	0	1.7	是
b	1.0	0	0.9	是
c	2.1	1	0.2	否
d	4.5	2	2.6	否

暴力 top-1 为 c，单表返回 b，Recall@1=0。精确重排无法找回没进入候选的 c。这是近似检索的正常失败，不应通过硬编码补回答案。

5 Basic 提交与验收

提交代码、数据/生成方法、桶与候选日志、暴力对照、Q1-Q5。验收看同一哈希是否用于建表和查询、候选是否真实缩小、漏检是否如实报告，不设置速度或召回门槛。

Advance：用多表换召回，测量实际代价

1 学习目标与难度

研究问题是：**怎样在查询时间、内存和 Recall@K 之间选择参数？** 数据小、向量化暴力搜索快时，LSH 未必更快，负结果同样有价值。

Goal	学习目标与完成要求	难度
G1	区分单表级联 k、多表 L 和返回数 K	easy
G2	固定库、开发查询与测试查询，建立精确答案	easy
G3	实现独立多表、候选并集去重和重排	medium
G4	记录 Recall@K、候选数、延迟、建表时间与内存	medium
G5	扫描参数，解释速度与召回的折中	medium
G6	从第 5 节选择一个方向阅读	按方向
G7	结果图、失败查询与 presentation	medium

G1-G7 均需完成，G6 只选一个。不要求达到固定加速倍数。

2 会用到的方法（同样先读这段）

k 让碰撞更严格，L 给多次机会。 一表内 k 个独立哈希都碰撞才进入候选；L 张表只需任一表碰撞，最后取并集。若某点单哈希碰撞概率为 p，理想独立情形下：

```
进入候选的概率 = 1 − (1 − p^k)^L
```

这是某个点对进入候选的概率，不是整个数据集 Recall@K 的直接数值保证，也不承诺常数查询时间。

三种成本都要算。 查询包括投影、查桶、去重、候选距离和 top-K 选择；建表另计。只比较距离计算次数会漏掉 Python 字典等开销。

编号	材料	建议用法
A1	Similarity Search via Hashing	主文献；阅读概率化候选筛选与实验思路
A2	v0 LSH 理论说明	参考 E2LSH 过程；以本说明的指标和保证边界为准
A3	v0 C++ 实现指南	阅读随机超平面、去重与 MNIST 组织

随机超平面主要对应角度/余弦相似度。若选这条扩展路线，应使用相容度量，或对非零向量 L2 归一化后利用余弦与欧氏距离排序的关系；零向量单独处理。

3 方法实现与过程展示（G2-G4）

建议库规模 5000-20000、维数 16-64；开发/测试各 100-200 个独立查询即可。也可用 MNIST 小子集，记录表示与归一化规则。

1. 建 L 张表，每张表独立抽投影和偏移；随机种子在整体开始设置，不能每表重置成相同值。
2. 查询所有同键桶，先去重，再精确重排；同距离按 ID 固定排序。

3. 用暴力搜索生成每个查询的真实 top-K 集 G，近似结果 R 的 Recall@K=|R∩G|/K。
4. 候选不足 K 时分母仍为 K；空结果记 0，并单列空桶率。
5. 只在开发查询上选 k、L、w，测试查询用于冻结后的最终评价。

固定 k 比较 L 时可复用同一组表的前缀；固定 L 比较 k 时可复用投影前缀，以减少无关随机差异。若启用暴力回退，必须另列回退率并把完整时间计入，不能与纯 LSH 混报。

4 比较方法并解释结果（G5 · medium）

可试 k=2/4/8、L=1/4/8，w 根据开发数据的距离尺度选取，不把某个常数当作通用值。报告至少 3 个建表种子的波动。

方法/参数	Recall@10	平均候选数	查询 P50/P95	建表时间/内存
暴力搜索	1.0	库大小	待测	无 LSH 索引
单表 LSH	待测	待测	待测	待测
多表 LSH	待测	待测	待测	待测

计时使用相同硬件、数据类型、批量方式和预热规则，重复查询计时但不复用结果缓存。画“召回-完整查询延迟”图，指出谁在相近召回下更快。提交至少 5 个漏检查询，解释桶边界、候选不足或高维距离集中等可能原因。

5 可选方向（G6 · 选择一个）

以阅读为主，不另加代码要求。

方向	难度	重点理解的问题	对应资料
G6-A：多探测	medium	查相邻桶如何用更多查询工作换召回？	A3
G6-B：随机超平面	medium	角度相似与欧氏相似何时一致？	A3
G6-C：增量插入	easy	保持哈希函数固定时，新向量如何入库？	A2

6 提交与讲解（G7 · medium）

提交索引与暴力实现、数据卡、查询切分、参数/种子、逐查询结果、时间与内存统计、折中曲线和失败案例。讲清“候选内精确”和“全库精确”的区别；分类正确率只能作为附加指标。

B 工作量安排与 Checkpoint

Basic：一周，约 10-20 小时

内容	预计用时
距离、哈希与手算	3-4 小时
暴力搜索和单表	4-6 小时
候选、边界与对照	2-4 小时
整理材料	1-2 小时
合计	约 10-16 小时

Checkpoint：手算漏检例子、真实候选日志、暴力对照、Q1-Q5。

Advance：在 Basic 基础上追加六周

时间	对应 Goal 与难度	主要任务	Checkpoint
第 1-2 周	G1、G2；G3：medium	固定数据与精确答案，实现多表	查询清单、去重正确、索引可复现
第 3-4 周	G4、G5：medium	参数扫描与完整计时	Recall/延迟图、内存、失败案例
第 5-6 周	G6 按方向；G7：medium	选读与汇报	解释概率、召回和实际成本

C 问题的来龙去脉与发展过程

从精确扫描到概率化候选。 高维相似检索需要同时考虑质量与成本。LSH 通过与相似度相关的碰撞概率减少待比较集合，允许用漏检换取潜在的速度收益。Similarity Search via Hashing

从一个桶到多次机会。 级联和多表分别控制碰撞严格程度与覆盖机会。它们改变候选分布，也增加投影、存储和合并开销。

从理论概率到工程证据。 某个点对的碰撞概率不能直接替代实际 Recall@K；距离计算减少也不保证总延迟降低。本项目用精确答案和完整计时把这两个层面连接起来。